Lecture 6: System calls

Professor G. Sandoval

06

NYU

Some slides adapted by G. Sandoval for CS3224, from Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved. Also some Slides by Brendan Dolan-Gavitt and Bryant and O'Hallaron, Computer Systems: A programmer's Perspective, Third Edition

Today

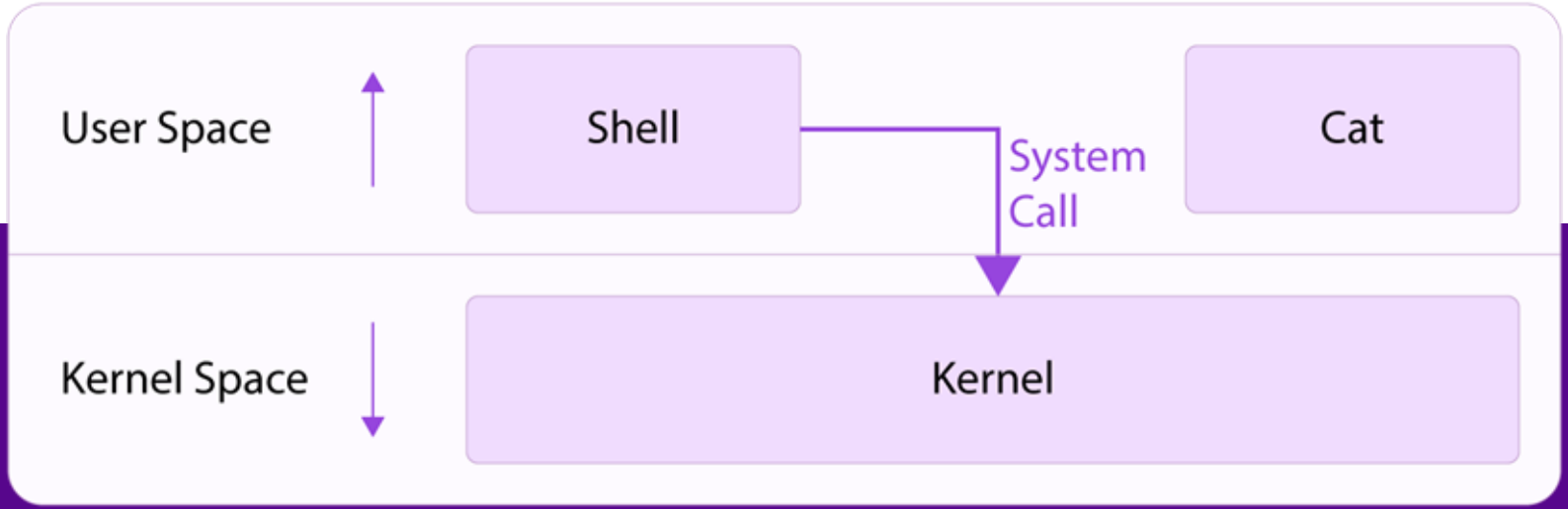
- **System Calls**
- XV6 System Calls Implementation
- Adding our own System Call

NYU

System Call Interface

- Once we are running a process in user mode that process must call into the kernel to access hardware
- **We do this using a system call**
- A system call is like a normal function call, but goes from **low-privilege** to **high-privilege** (user->kernel)

Making a System Call



System call	Description
fork()	Create Process
exit()	Terminate current process
wait()	Wait for a child process to exit
kill(pid)	Terminate process pid
getpid()	Return current process's id
sleep(n)	Sleep for n seconds
exec(filename, *argv)	Load a file and execute it
sbrk(n)	Grow process's memory by n bytes
open(filename, flags)	Open a file; flags indicate read/write
read(fd, buf, n)	Read n bytes from an open file into buf

System call	Description
write(fd, buf, n)	Write n bytes to an open file
close(fd)	Release open file fd
dup(fd)	Duplicate fd
pipe(p)	Create a pipe and return fd's in p
chdir(dirname)	Change the current directory
mkdir(dirname)	Create a new directory
mknod(name, major, minor)	Create a device file
fstat(fd)	Return info about an open file
link(f1, f2)	Create another name (f2) for the file (f1)
unlink(filename)	Remove a file

How many system calls do you think Linux has?

~50 — 0%

~ 300 — 0%

~ 1000 — 0%

None of the above — 0%

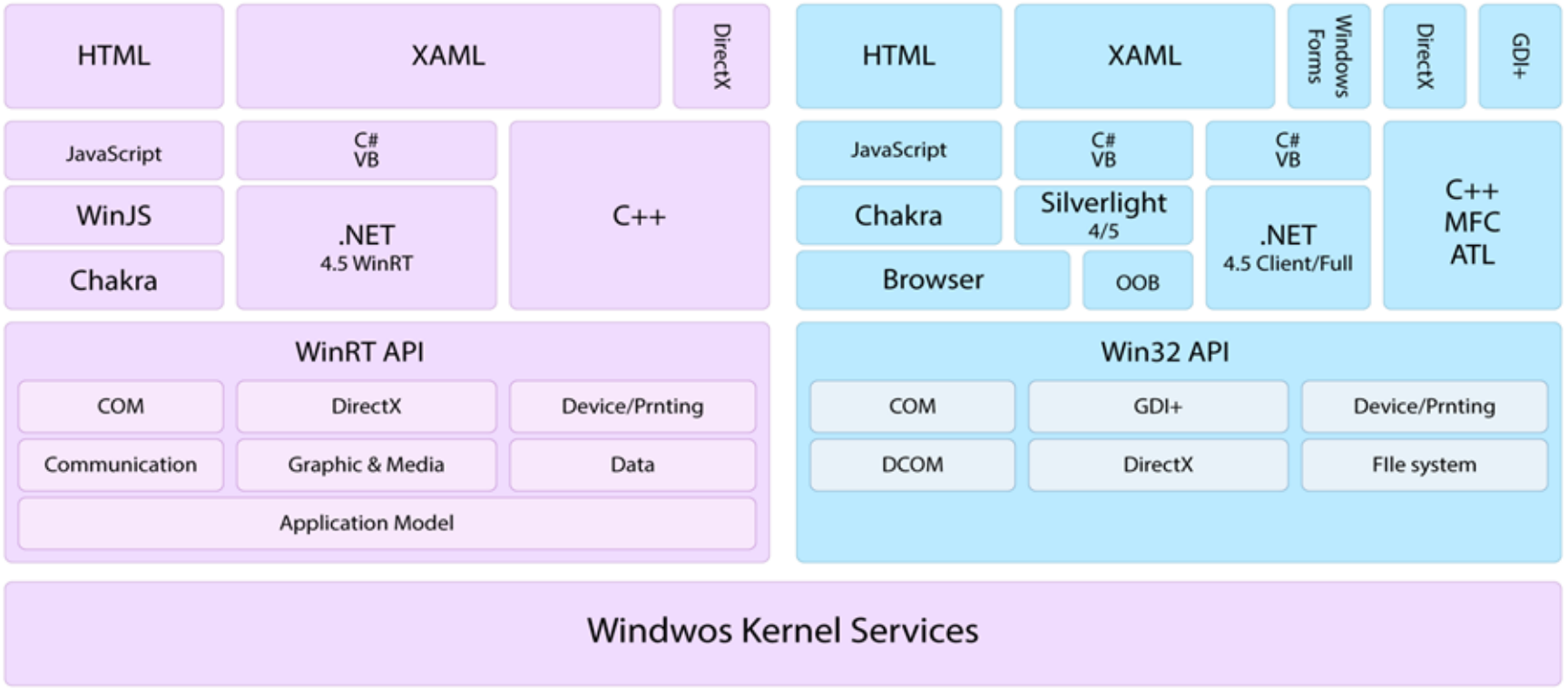
Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Real-World System Calls

- Linux is a UNIX-like operating system, so the concepts are similar
- But it has many more calls – **389** as of kernel version 5.19
- Interface is **public** – user programs can use it directly
- Many more than xv6!

Real-World System Calls

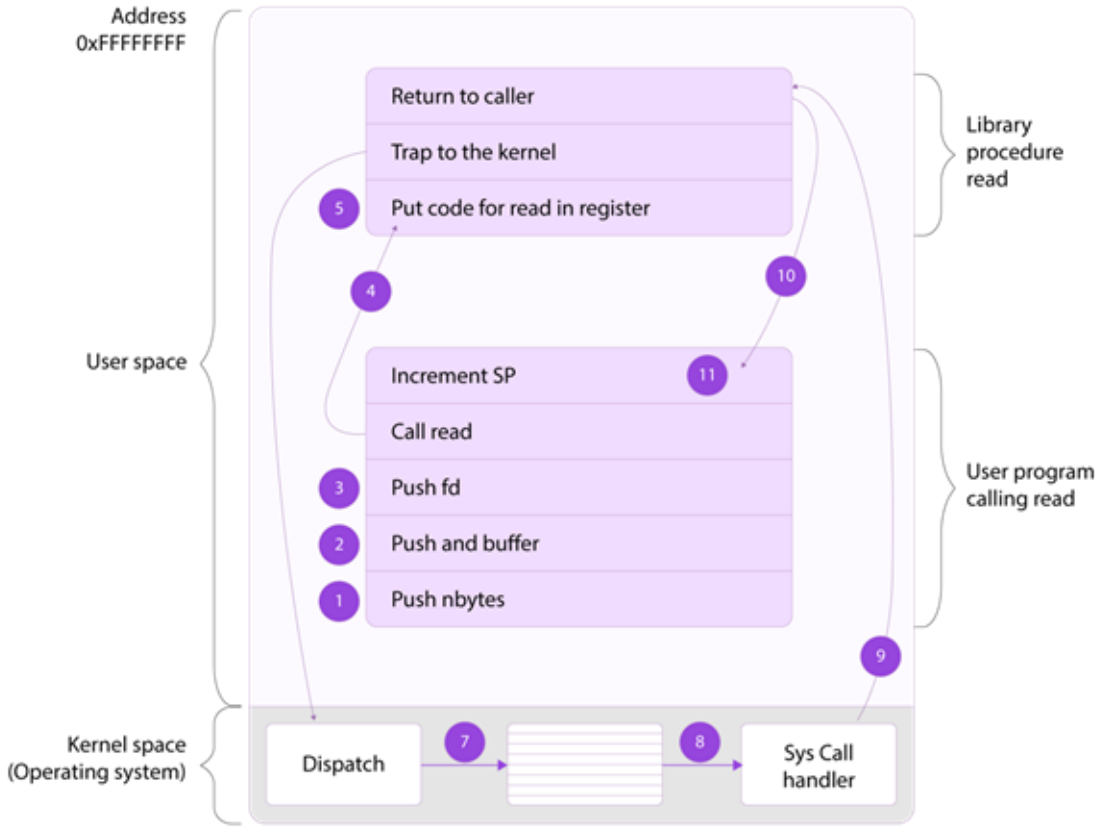
- Windows has even more – close to **2000** as of Windows 11
- The majority of these (1,036) are GUI-related
- The system call interface is **private**
- All calls go through a (much larger) set of **user-space libraries** that are stable (the Win32 API)


HTML
XAML
DirectX
JavaScript
C#
VB
WinJS
.NET
4.5 WinRT
Chakra
C++
WinRT API
COM
DirectX
Device/Prnting
Communication
Graphic & Media
Data
Application Model
HTML
XAML
Windows Forms
DirectX
GDI+
JavaScript
C#
VB
C#
VB
Chakra
Silverlight
4/5
.NET
4.5 Client/Full
C++
MFC
ATL
Browser
OOB
Win32 API
COM
GDI+
Device/Prnting
DCOM
DirectX
FIle system
Windwos Kernel Services

Demo minitrace

- strace /opt/homebrew/bin/gcat /etc/hosts

System Calls



The 11 steps in making the system call *read(fd, buffer, nbytes)*.

System Call Mechanism

Have an instruction that initiates the call

- Specify **which** call we want
- **Save** the process's **current** state and **restore** it when we return

Setup for execution in the kernel

- Choose a place in the kernel to start executing
- Have the kernel retrieve the system call arguments

Do all this securely, without breaking isolation between user and kernel space

NYU

SysCall, Exceptions & Interrupts

- **System Call:** Way for user program to ask for OS service
- **Exception:** Illegal program action. Eg. Divide by zero, access memory for PTE not present, etc..
- **Interrupt:** Signal generated by **hardware** device. Eg. Clock chip, keyboard, etc.

Hardware Handling

- X86 uses only one mechanism for all three: **INT** instruction
- Therefore mechanism is the same for handling all three in the OS !
- That’s why XV6 handles them in **trap.c**

Code: Trap.c

Crossing the User / Kernel boundary

Four protection modes (0 -3) mostly 0 & 3 used

Kernel mode: Ring 0

User mode: Ring 3

“Ring 0” and “Ring 3”

OS: “event driven”

- Events causing mode switches

  Systems Calls: issued by user process to request system services

  Exceptions: Illegal instructions (e.g. division by 0)

  Interrupts: raised by devices to get OS attention

- **Often handled using the same hardware mechanism:** interrupt

  Also called trap

NYU

Interrupt view of the CPU

```
while (fetch next instruction) {
   run instruction;
   if (there is an interrupt) {


      process interrupt


   }
}
```

NYU

Interrupt view of the CPU

```
while (fetch next instruction) {
    run instruction;
    if (there is an interrupt) {
        switch to kernel stack if necessary
        save CPU context and error code if any
        find OS-provided interrupt handler
        jump to handler
        restore CPU context when handler returns
    }
}
```

NYU

Interrupt view of the CPU

```
while (fetch next instruction) {
    run instruction;
    if (there is an interrupt) {
        switch to kernel stack if necessary
        save CPU context and error code if any
        find OS-provided interrupt handler
        jump to handler
        restore CPU context when handler returns
    }
}
```

Q1: What CPU Context Save and restore?

Q2: What does the Handler do?

Q3: Why do we switch stacks?

NYU

Interrupt view of the CPU

```
while (fetch next instruction) {
    run instruction;
    if (there is an interrupt) {
        switch to kernel stack if necessary
        save CPU context and error code if any
        find OS-provided interrupt handler
        jump to handler
        restore CPU context when handler returns
    }
}
```

Q4: How does hardware find OS-provided interrupt handler?

Q5: Why switch stack?

Q6: What CPU context to save and restore?

Q7: What does handler do?

NYU

The int Instruction

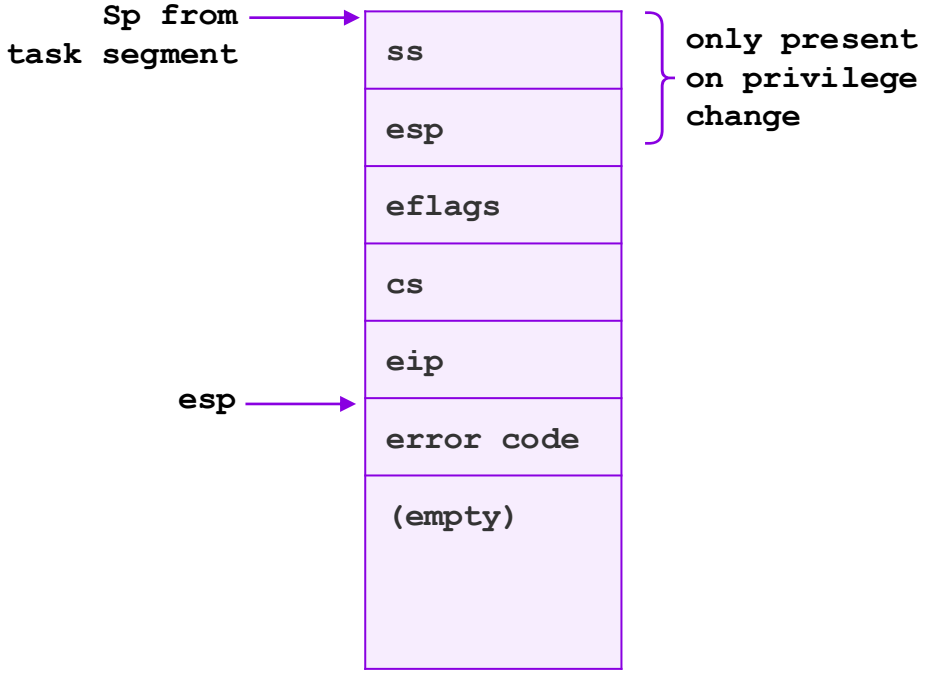
"int n" does a lot of work

- **Fetch the nth descriptor** from the IDT, where n is the argument of int.
- **Check that CPL in %cs is <= DPL,** where DPL is the privilege level in the descriptor.
- Save %esp and %ss in a CPU-internal registers
- Load %ss and %esp from a task segment descriptor.

The int Instruction

- Push %ss.
- Push %esp.
- Push %eflags.
- Push %cs.
- Push %eip.
- Clear some bits of %eflags.
- Set %cs and %eip to the values in the descriptor.

Stack after int Instruction



Kernel stack after an int instruction.

Why So Complicated?

Mostly to ensure **protection**

- CPL check **prohibits anything** but a user process from making any other interrupts.
- Kernel reads some information off the stack; arbitrary user stacks might be **incorrect** (point to nonexistent memory) or **malicious**.

Today

- System Calls
- **XV6 System Calls Implementation**
- Adding our own System Call

NYU

initcode.S

Last time we saw the system's first system call

```
# exec(init, argv)
.globl start
start:
  pushl $argv
  pushl $init
  pushl $0  // where caller pc would be
  movl $SYS_exec, %eax
  int $T_SYSCALL
[...]
# char init[] = "/init\0";
init:
  .string "/init\0"
```

NYU

Trap Frames

Once we are in the kernel, we save more registers (alltraps, trapasm.S)

```
alltraps:
  # Build trap frame.
  pushl %ds
  pushl %es
  pushl %fs
  pushl %gs
  pushal
```

Pushes all general-purpose registers onto the stack

Together with the registers the CPU saved for us, this makes up a trap frame

NYU

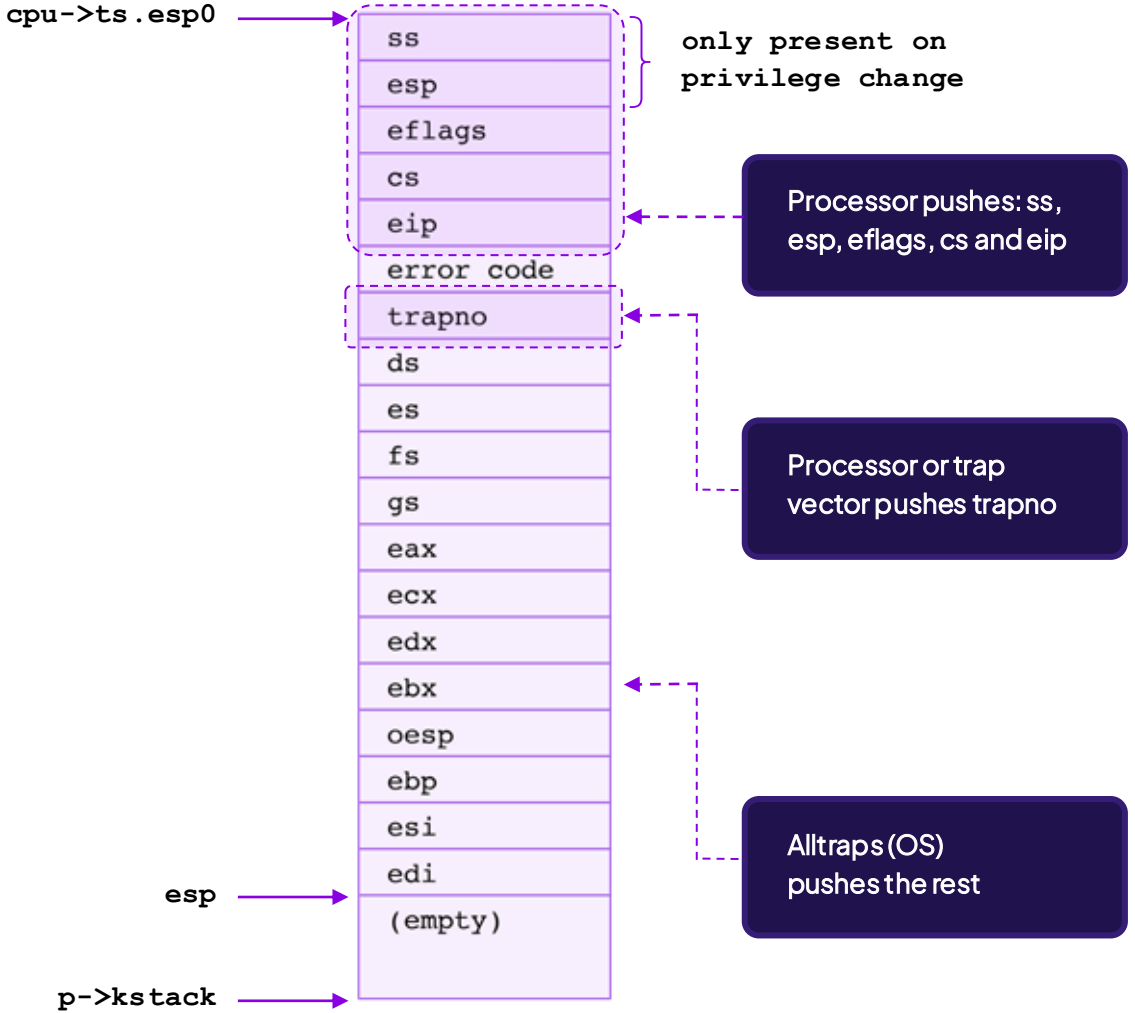


Figure 3-2. The trapframe on the kernel stack

Trap Frames

```
// Layout of the trap frame built on the stack by the
// hardware and by trapasm.S, and passed to trap().
struct trapframe {
  // registers as pushed by pusha
  uint edi;
  uint esi;
  uint ebp;
  uint oesp;      // useless & ignored
  uint ebx;
  uint edx;
  uint ecx;
  uint eax;

  // rest of trap frame
  ushort gs;
  ushort padding1;
  ushort fs;
  ushort padding2;
  ushort es;
  ushort padding3;
  ushort ds;
  ushort padding4;
  uint trapno;
...
```

NYU

Trap Frame

- By capturing all of this information in the trap frame structure, we can restore the CPU state exactly when we return from the system call

Trap Handler

- After we've set up our trap frame, we call the trap() function:

```
# Call trap(tf), where tf=%esp
pushl %esp
call trap
addl $4, %esp
```

Trap Handler – trap.c

```
void
trap(struct trapframe *tf)
{
  if(tf->trapno == T_SYSCALL){
    if(proc->killed)
      exit();
    proc->tf = tf;
    syscall();
    if(proc->killed)
      exit();
    return;
  }

  switch(tf->trapno){
  case T_IRQ0 + IRQ_TIMER:
    if(cpu->id == 0){
      acquire(&tickslock);
      ticks++;
[...]
```

NYU

Inside the System Call

- After setting up the trap frame, we call the syscall() function
- **syscall**() examines %**eax** in the trap frame to find out what system call to execute
- Inside the system call, things look exactly the same as a normal function call
- But we have extra privileges since we are in kernel mode;

  e.g. we can talk to hardware directly

kill(pid)

```
void
syscall(void)
{
  int num;

  num = proc->tf->eax;
  if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
    proc->tf->eax = syscalls[num]();
  } else {
    cprintf("%d %s: unknown sys call %d\n",
            proc->pid, proc->name, num);
    proc->tf->eax = -1;
  }
}

int
sys_kill(void)
{
  int pid;

  if(argint(0, &pid) < 0)
    return -1;
  return kill(pid);
}
```

NYU

Retrieving Arguments

- We changed stacks after leaving user mode
- But arguments are still stored on the user stack!
- Solution: use the %esp value saved in the trap frame
- Arguments are at %esp+4+(4*arg_no)

Retrieving Arguments

```
// Fetch the nth 32-bit system call argument.
int
argint(int n, int *ip)
{
  return fetchint(proc->tf->esp + 4 + 4*n, ip);
}

int
fetchint(uint addr, int *ip)
{
  if(addr >= proc->sz || addr+4 > proc->sz)
    return -1;
  *ip = *(int*)(addr);
  return 0;
}
```

NYU

Protection

- Note the check in fetchint to make sure the pointer is not outside **proc->sz**
- In user mode, we can rely on the paging hardware to disallow access to anything outside of process's memory
- But in kernel-mode **we must do explicit checks**, because the kernel has access to all memory
- Similar checks for getting a **pointer (argptr)** and getting a **string (argstr)**

Returning to Userspace

- syscall() put the return value in proc->tf->eax
- So when we get back to userspace, we will have our return value in %eax

```
.globl trapret
trapret:
  popal
  popl %gs
  popl %fs
  popl %es
  popl %ds
  addl $0x8, %esp  # trapno and
errcode
  iret
```

NYU

Today

- System Calls
- Int Instruction
- XV6 System Calls Implementation
- **Adding our own System Call**

NYU

Adding a System Call

- From this, it should be easy now to **add a system call**
- We don't have to alter the mechanism, just **extend** the list and **implement** our function

Hello World

- We will **add a system call** that just prints hello world an a user-provided number
- First we need to **add our call to the list** in syscall.c

```
extern int sys_write(void);
extern int sys_uptime(void);
extern int sys_hello(void);
static int (*syscalls[])(void) =
{
[SYS_fork]    sys_fork,
[SYS_exit]    sys_exit,
[SYS_wait]    sys_wait,
...
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
[SYS_hello]   sys_hello,
};
```

NYU

Hello World

- Next, assign it a number in **syscall.h**:

```
#define SYS_mkdir   20
#define SYS_close   21
#define SYS_hello   22
```

- And give it a prototype in **user.h**:

```
int dup(int);
int getpid(void);
char* sbrk(int);
int sleep(int);
int uptime(void);
int hello(int);
```

NYU

usys.S

Add it to **usys.S**, which generates the user-space assembly code for it

```
#define SYSCALL(name) \
  .globl name; \
  name: \
    movl $SYS_ ## name, %eax; \
    int $T_SYSCALL; \
    ret

SYSCALL(fork)
SYSCALL(exit)
...
SYSCALL(uptime)
SYSCALL(hello)
```

NYU

Implementation

Finally we add the implementation somewhere **(e.g. sysproc.c)**

```
int
sys_hello(void) {
    int n;
    if(argint(0, &n) < 0)
        return -1;
    cprintf("Hello world %d\n", n);
    return 0;
}
```

NYU

Testing Our New Call

```c
#include "types.h"
#include "stat.h"
#include "user.h"

int main(void) {
    hello(5);
    exit();
}
```

NYU

NOW YOUR TURN

Write a system call that prints your netid

Breakout

Real-World: A Newer Mechanism

- Older versions of Linux and Windows did use interrupts for system calls
- Nowadays most systems don't use them
- Interrupts are very heavyweight, slow
- Instead we have sysenter and sysexit
- These do much less work – mostly just switches to kernel mode, sets CS, ESP, and EIP
- Up to the kernel to decide how much it wants to save from user mode